



I want to build something like the turbo tax of completing occumeds forms using replit



Thought for 9s >

Yes — and Replit is actually a pretty good place to build an internal **"TurboTax for Occu-Med forms"** style app because it can handle app generation, database setup, and deployment in one place. Replit's current platform explicitly supports AI-built web apps, built-in SQL databases, authentication patterns, and direct deployment workflows.

replit +3

What you want is not really "a form filler."
It should be an **interview engine**.

The user experience should feel like this:

1. Pick exam type: Deployment, Traditional, Dental, Labs Only, etc.
2. Answer one question at a time in plain English.
3. The system hides irrelevant sections.
4. It asks intelligent follow-ups only when triggered.
5. It checks for missing required fields before moving on.
6. At the end, it generates a clean packet-ready output and a staff review summary.

That is exactly why the TurboTax comparison works.

What the app should actually do

Think of it as 5 layers:

1. Intake wizard

A guided questionnaire instead of dumping DD forms on the user.

2. Rules engine

This is the brain.

Example:

- If exam type = DoD deployment, include DD 2807-1, DD 2808, DD 2215, etc.
- If history of hypertension = yes, ask follow-up questions
- If audiogram is required, prompt for examiner + calibration date
- If dental only, hide unrelated medical sections

3. Packet completion tracker

A visual progress bar:

- Demographics complete
- Medical history complete
- Required signatures complete
- Required attachments complete
- Outstanding issues flagged

4. Staff review screen

For ExamQA / Network / SMEs:

- See entered answers
- See missing components
- See contradictions
- See recommended next actions

5. Output generator

Generate:

- printable summary
- structured case notes
- exportable PDF packet data
- internal checklist

Best MVP to build first

Do **not** start by trying to fully auto-complete every PDF.

Start with this MVP:

Version 1

- login
- create new case
- choose exam type
- guided interview
- required-field validation

- internal completeness dashboard
- final case summary page

Version 2

- reusable templates by packet type
- RDQA-style follow-up question trees
- role-based views for Scheduling / Harvesting / ExamQA / SME
- PDF autofill/export

Version 3

- AI-assisted wording help
- document upload + extraction
- contradiction detection
- "missing from packet" flags
- clinic-facing simplified request sheet

That sequence is more realistic and much less likely to collapse.

What stack I'd use in Replit

A solid Replit build for this would be:

- **Frontend:** React
- **Backend:** Node/Express or Next.js API routes
- **Database:** Replit SQL database
- **Auth:** simple email/password or Google login
- **PDF/export:** pdf-lib or similar
- **AI layer:** optional later, for summaries and follow-up suggestions

Replit's current docs say its built-in SQL database can be added directly through the platform/Agent workflow, and its app builder is designed for building and deploying dynamic web apps. [replit +2](#)

The most important design choice

Do **not** build it around the forms themselves.

Build it around a **question schema**.

Each question should live in a config object like:

- question id
- label

- answer type
- required or not
- which packet types it belongs to
- what answer triggers follow-ups
- which output field it maps to
- what warning appears if blank/inconsistent

That way, when Occu-Med changes a form or process, you update rules instead of rebuilding the app.

Example of how one section should work

Instead of showing:

"Do you have hypertension?"

The app asks:

- Have you ever been diagnosed with high blood pressure?
- Are you currently taking medication for it?
- Do you have any work restrictions?
- Have you ever been hospitalized because of it?
- Is your condition stable according to your provider?
- Do you need accommodations or modifications?

Then the system decides:

- no issue
- needs clarification
- likely RDQA follow-up
- sufficient for review

That is the real value.

What makes this valuable for Occu-Med

This could reduce a lot of the current friction you keep running into:

- staff not knowing which questions to ask
- inconsistent follow-up
- packet omissions
- confusion over what is required for which exam
- too much knowledge living in people's heads

- new employees not being trained on all the nuance

So the app becomes:

- a training tool
- a workflow standardizer
- a packet quality checker
- a case-prep assistant

What I would name it

A few names that fit:

- **Occu-Guide**
- **PacketPath**
- **OccuFlow**
- **ExamPilot**
- **FormWise**
- **ClearPath by Occu-Med**

My pick: **ExamPilot** or **PacketPath**.

Replit prompt to start building it

Paste this into Replit Agent as your starting build prompt:

Build an internal web app called "PacketPath" for Occu-Med.



Purpose:

This app should work like TurboTax for completing occupational health exam work

Core requirements:

- Secure login
- Dashboard showing all cases
- Create new case
- Choose exam type:
 - Deployment packet
 - Traditional packet
 - Dental only
 - Labs only
- Guided multi-step questionnaire
- Show one question at a time
- Conditional logic for follow-up questions
- Required field validation
- Progress bar by section
- "Missing items" warnings

- Review page before final submission
- Admin area to manage question templates and packet rules
- Save all data to a SQL database
- Clean, modern UI with a professional medical/admin look
- Mobile responsive but optimized for desktop

Data model:

- Users
- Cases
- Exam types
- Question templates
- Answers
- Rule mappings
- Completion status
- Review flags

Features:

- Each question should support:
 - text
 - yes/no
 - dropdown
 - date
 - number
 - multi-select
- Questions can trigger follow-up questions based on answers
- Questions can belong to one or more exam types
- Review screen should show:
 - answered questions
 - unanswered required questions
 - flagged contradictions
 - recommended next steps
- Add role types:
 - Admin
 - ExamQA
 - Reviewer

Design:

- Professional clean vector-style medical workflow interface
- Dark navy and soft neutral color palette
- Rounded cards and polished internal-tool UI
- Clear section labels and minimal clutter

Tech:

- Use React frontend
- Use Node/Express backend
- Use Replit SQL database
- Seed the app with example exam types and sample questions

Also generate:

- a simple database schema
- sample question logic
- a starter admin panel for editing questions

My blunt advice

The mistake would be trying to build:

- full PDF automation first
- AI medical reasoning first
- every packet type first
- every rule first

Build the **guided decision tree + completeness checker** first.

That alone could already be useful.

What I think you should build first-first

The absolute first usable version should only cover:

- one packet type
- one role
- one ruleset
- one final summary output

For example:

Deployment packet assistant for ExamQA staff only

That is much more likely to become real.

If you want, I can map the actual screens, database tables, and question logic for your first Replit version next.

    ...  Sources

So I am on the free tier .. i don't have extra money for the full version so the prompts will need to be surgical and thorough and address all aspects

I wish there was a way we could build it to be usable with anymore ...

Like what it would understand function, and the logic of what it's doing that if I give it a form, it's going to ask those questions in one by one for a minute if it's a no move onto the next one if it's a yes, ask this follow up question that way no matter what the form is I could just upload it and do it manually like configure it manually and then after that, it knows what to do no matter how many times it's refilled by someone

Thought for 13s >

Yes — that is the **right way** to think about it.

Do **not** build “an app for DD2807” or “an app for one packet.”

Build a **form logic engine** where you can teach the system:

- what each question is
- what type of answer it expects
- whether it is required
- what happens if the answer is **No**
- what follow-up happens if the answer is **Yes**
- where that answer should be stored
- how to reuse that same logic every time the same form gets filled again

That is much smarter, especially on Replit’s free plan, which currently gives limited Agent intelligence, free daily Agent credits, and 1 published app. replit +1

What you should build instead

You want a system with **2 modes**:

1. Builder mode

This is where **you** configure a form once.

You upload or define a form and manually teach the app:

- Section name
- Question text
- Answer type
- Required or optional
- Trigger logic
- Follow-up logic
- Display order
- Notes/help text
- Output mapping

Example:

Question: Do you have a history of hypertension?

Type: Yes/No

If No: move to next question

If Yes: ask:

- Are you taking medication?
- Is it controlled?
- Any restrictions?
- Any hospitalizations?
- Provider clearance available?

Once you save that logic, the app remembers it forever.

2. Filler mode

This is what staff use later.

They do **not** see the raw form logic.

They just get the guided interview:

- one question at a time
- only relevant follow-ups
- smart skipping
- progress tracking
- final review

That is the reusable system.

The key insight

Your app should not “understand forms” like magic.

It should understand a **schema**.

That means every form becomes structured like this:

- Form
- Sections
- Questions
- Rules
- Follow-up branches
- Completion checks

So instead of hardcoding every form, you create a **universal question engine**.

That means later you can add:

- DD 2807-1
- DD 2808
- DD 2215
- Occu-Med questionnaire
- dental forms
- client-specific forms

without rebuilding the whole app.

The cheapest realistic version

On free tier, do not ask Replit to build the whole dream at once.

Have it build this exact order:

Phase 1

Build a working engine with:

- login optional or skipped for now
- create form template
- add sections
- add questions

- set yes/no branching
- run the interview
- save answers
- show summary

Phase 2

Add:

- required field validation
- edit existing templates
- duplicate template
- completion percentage
- reviewer notes

Phase 3

Add:

- PDF field mapping
- export summary
- contradiction flags
- attachments/doc upload

That is much more survivable on limited credits.

The data structure you want

Tell Replit to think in terms of these tables or objects:

forms

- id
- name
- description

sections

- id
- form_id
- title
- order

questions

- id
- section_id
- question_text
- help_text
- answer_type
- required
- order

rules

- id
- question_id
- trigger_value
- action_type
- target_question_id

form_responses

- id
- form_id
- respondent_name
- created_at

answers

- id
- response_id
- question_id
- answer_value

That structure is the heart of the whole product.

How the logic should work

The rule engine can stay simple at first.

Basic rule types:

- show next question
- show follow-up question

- skip question
- mark section incomplete
- flag for review

Example logic:

- If answer = "No" → next normal question
- If answer = "Yes" → reveal child follow-up questions
- If answer is blank and required → block completion
- If answer conflicts with another answer → add review flag

You do **not** need AI for this part.

This is plain logic.

That is good news because it makes it cheaper and more stable.

Best MVP concept

Call it something like:

- OccuFlow Builder
- Packet Logic
- FormPath
- OccuTax
- QuestionPilot

My favorite for what you described: **FormPath**

Because it really is "define the path once, reuse forever."

Surgical Replit strategy

Because your credits are limited, you should not give one giant messy prompt.

Use **one master prompt**, then smaller follow-up prompts.

Master prompt for Replit

Paste this first:



Build a web app called FormPath.

Purpose:

This app should act like a reusable form logic engine for Occu-Med style workfl

Core concept:

There are two modes:

1. Builder Mode

An admin can create a form template by defining:

- form name
- sections
- questions
- answer type
- whether the question is required
- help text
- display order
- conditional branching rules
- follow-up questions triggered by answers

2. Filler Mode

A user fills out a selected form through a guided one-question-at-a-time workfl

- If answer is No, move to next applicable question
- If answer is Yes, show the configured follow-up questions
- Skip irrelevant questions automatically
- Save all answers
- Show progress
- Show final review summary

Must-have features:

- Dashboard with list of form templates
- Create new form template
- Edit existing template
- Add sections to a form
- Add questions to a section
- Supported answer types:
 - short text
 - long text
 - yes/no
 - number
 - date
 - dropdown
- Add simple conditional logic:
 - when answer equals X, show target question
 - when answer equals X, skip target question
- Guided interview runner for filling out forms
- Review page showing all answers and missing required fields

Important:

Do NOT hardcode this app for a specific form.

It must be schema-driven and reusable for unlimited forms.

Data model:

- forms
- sections
- questions
- rules
- form_responses
- answers

UI:

- clean professional admin tool
- simple navigation
- left sidebar
- clean cards
- minimal clutter
- optimized for desktop

Tech:

- React frontend
- Node/Express backend
- SQLite or Replit database
- simple local persistence is okay for MVP

Please generate:

1. project structure
2. database schema
3. working builder mode
4. working filler mode
5. sample seeded form template to demonstrate branching logic

Then do follow-up prompts one by one

After it builds the base, use smaller prompts like these:

Prompt 2: improve question builder

Improve Builder Mode so that when creating a question, I can also set:

- help text
- required yes/no
- display order



- parent question
- trigger answer value

Also make the question editor cleaner and easier to use.

Prompt 3: improve branching

Improve the rule engine so yes/no branching works reliably. 

Behavior:

- If a question has child follow-up questions triggered by "Yes", only show the
- If answer is No, skip those follow-up questions and continue normally.
- Support nested follow-up questions.

Prompt 4: final review

Add a final review page that shows: 

- all answered questions grouped by section
- all required unanswered questions
- completion percentage
- clear warning banners for missing required answers

Prompt 5: duplicate templates

Add a duplicate form template feature so I can clone an existing form and modify it. 

What to avoid saying to Replit

Avoid prompts like:

- "Build the TurboTax of forms"
- "Make it smart"
- "Make it understand PDFs"
- "Make it AI-powered"

Those sound good, but they are too vague and burn credits.

Instead use:

- exact screens

- exact tables
 - exact logic
 - exact behaviors
-

What about uploading a form?

For now, think of “uploading a form” in the cheap version as:

- upload the PDF for reference
- manually create the schema from it
- save that schema as a reusable template

That is realistic.

Later, once the engine works, you can add:

- PDF preview side-by-side
- manual field mapping
- maybe AI-assisted question extraction

But **not first**.

The smartest cheap architecture

Your app should separate:

A. Form template

The permanent logic you build once

B. Form session

A single person filling it out this time

That way:

- template stays untouched
 - each new exam creates a new response session
 - staff can reuse the same template over and over
-

If you want it to be usable for “any form”

Then the non-negotiables are:

- dynamic sections
- dynamic questions
- dynamic rules
- reusable templates
- no hardcoded form names
- no hardcoded follow-up logic

If those are done right, you only need to “teach” each form once.

My honest recommendation

Your first working version should only prove these 4 things:

1. You can create a template
2. You can define yes/no branching
3. A user can fill it out one question at a time
4. The app saves and reviews the answers

If that works, then the whole idea is real.

If you want, I can write you the **next even more surgical Replit prompt set**, broken into tiny chunks so you can paste them one at a time without wasting credits.

      Sources

OK, I get what you're saying but I still want them to see the form as they're filling it so it does need to be formed specific to an extent. It just has to allow me to train it on different forms.

Yes — that makes sense, and it actually changes the product in a good way.

What you want is **not** just a question wizard.

You want a **form-guided interview system**.

So the app should do both at the same time:

- show the actual form on screen
- guide the user through it one question at a time
- highlight the field they are currently answering
- ask follow-up questions when needed
- let you "train" new forms so the same behavior works again later

That is a much better product than a blind questionnaire.

The right model

Think of it like this:

Layer 1: Form viewer

The user sees the real form:

- PDF or image of the form
- section by section
- highlighted current question/field

Layer 2: Interview logic

The app asks:

- "Answer this field now"
- if no, continue
- if yes, open configured follow-ups
- if required and blank, do not allow completion

Layer 3: Form training

You teach the app once:

- what each visible form field means
- what question text to ask
- what answer type it expects
- what follow-ups belong to it
- where the answer belongs on the form
- what area of the form should highlight when asked

That is the real architecture.

So yes, it should be form-specific and reusable

The important distinction is:

It should **not** be hardcoded by developer for each form.

It should be **configurable by you** for each form.

So the system becomes:

- generic engine underneath
- form-specific templates on top

That way you can train:

- DD 2807-1
- DD 2808
- DD 2215
- Occu-Med questionnaire
- any custom client form

without rebuilding the app every time.

What the app should feel like

Imagine the screen split into 2 panes:

Left side

The actual form preview

Right side

The guided interview panel:

- current question
- answer options
- help text
- follow-up prompts
- next / back buttons

When a question is active, the matching box or area on the form gets highlighted.

That is probably the best UX for what you want.

What "training a form" should mean

When you add a new form, you should be able to:

1. Upload the PDF or image of the form
2. Create named fields for each part
3. Draw or assign the field's location on the form
4. Define the question text
5. Choose answer type
6. Mark required vs optional
7. Set logic rules
8. Save it as a reusable template

Then every time someone uses that form later, the app already knows what to do.

This is the exact structure you need

A. Form template

Stores:

- form name
- uploaded PDF/image
- pages
- field coordinates
- sections
- questions
- rules

B. Field mapping

For each form field:

- field id
- label
- page number
- x/y position
- width/height
- linked question id

C. Question logic

For each question:

- question text
- answer type
- required

- help text
- follow-up triggers
- child questions

D. Response session

For each time someone fills the form:

- selected template
- answers
- completion status
- flagged issues

Cheapest smart version to build first

Do not start with drag-and-drop mapping if you are on limited credits.

Start with a simpler training workflow:

Version 1

- upload form PDF
- show it in a viewer
- manually create questions in a list
- for each question, assign:
 - page number
 - approximate label
 - highlight note
- guided interview runs beside form
- no exact field box highlighting yet, just section highlighting or field labels

Version 2

- click on PDF to place field boxes
- store exact coordinates
- highlight exact question location

Version 3

- export answers into the mapped PDF

That is the safer path.

Best first version of the UI

Screen 1: Templates dashboard

- all saved forms
- create new template
- edit template
- run template

Screen 2: Template builder

- upload form
- add section
- add questions
- define branching
- assign each question to page/field label

Screen 3: Fill mode

- left: form preview
- right: one active question at a time
- next/back
- progress bar
- follow-up logic

Screen 4: Review mode

- all answers by section
- missing required answers
- flagged follow-ups
- ready for export

The exact prompt direction you should give Replit

You need to stop describing it as only "TurboTax for forms."

Describe it like this:

Build a web app called FormPath.



Purpose:

This app should let me upload and configure different forms, then guide users t

Core idea:

The app must support reusable form templates. Each template represents a speci

Main modes:

1. Template Builder Mode

- Upload a form PDF or image
- Create sections for the form
- Add questions linked to that form
- For each question, define:
 - question text
 - answer type
 - required yes/no
 - help text
 - display order
 - follow-up logic
 - page number on the form
 - field label or mapped area reference
- Save the form as a reusable template

2. Fill Mode

- Show the actual form on the left side of the screen
- Show a guided one-question-at-a-time interview on the right side
- As each question is asked, highlight the related part of the form
- If answer is No, move to the next applicable question
- If answer is Yes, show configured follow-up questions
- Skip irrelevant questions automatically
- Save all answers during the session
- Show progress by section

3. Review Mode

- Show all answers grouped by section
- Show missing required fields
- Show flagged follow-up areas
- Allow editing before final save

Important requirements:

- Do NOT hardcode one specific form
- The app must support many form templates
- Each form template should store both visual form information and interview logic
- The interview logic must be reusable every time that form is filled out again

Suggested data model:

- form_templates
- form_pages
- form_fields
- sections
- questions
- rules
- response_sessions

- answers

UI requirements:

- Split-screen layout in Fill Mode
- PDF or image viewer on the left
- Guided interview panel on the right
- Clean professional internal-tool design
- Desktop-first

Tech:

- React frontend
- Node/Express backend
- SQLite or Replit database
- Keep MVP simple and reliable

For the MVP:

- It is okay if field highlighting is basic at first
- It is okay if mapping is manual instead of automated
- Focus on making reusable template logic work correctly

Even more surgical first build request

If you want to save credits, make the first ask even narrower:

Build only the MVP foundation for a reusable form-guided interview app.



MVP requirements:

- Create form templates
- Upload one PDF for each template
- Add questions manually
- Add yes/no branching rules
- Run the form in split-screen mode:
 - left side shows the uploaded form
 - right side shows one question at a time
- Save answers
- Show review page

Do not build PDF autofill yet.

Do not build AI extraction yet.

Do not build drag-and-drop mapping yet.

That version is much more likely to succeed on free tier.

Best product framing

The sentence you should keep in your head is:

"I am building a reusable form-template engine with a split-screen guided completion workflow."

That is exactly what this is.

Not:

- a static PDF tool
- a generic chatbot
- a hardcoded single-form app

My blunt recommendation

Your first successful version should prove only this:

- you can upload one form
- you can define questions for it
- the user can see the form while answering
- yes/no branching works
- the answers save correctly

Once that works, the rest is layering.

If you want, I can write the **tiny step-by-step Replit prompts in exact order**, with each prompt designed to use as few credits as possible.



Why are u complicating it with a split screen? Lets just do this once and do it right and complete

You're right to push back — split screen is not required.

If your goal is:

👉 **"Train once → reuse forever → works for any form → feels like TurboTax"**

Then the **cleanest, most powerful version** is actually this:

The *right* version (simple + complete)

Forget split screens.

Build a system that:

1. Ingests a form (PDF)
2. Lets you define logic ON TOP of it
3. Runs as a guided interview
4. Outputs a completed version (or structured data)

The form becomes the *reference layer*, not the UI.

The architecture you actually want

1. Template = Form Brain

Each form you "train" becomes a **template object**:

- form file (PDF)
- list of questions
- answer types
- rules
- follow-ups
- field mappings

Once saved → reusable forever.

2. Interview Engine (the core)

This is the only thing that matters.

The app runs like:

- Ask question 1
- If No → go to next
- If Yes → open follow-ups
- Continue until complete
- Validate required fields
- Show review

That's it.

No UI complexity needed.

3. Optional (later): Output layer

- map answers → PDF fields
- or generate structured summary

But **NOT** needed for MVP

What you ACTUALLY want (your idea distilled)

You said something really important:

“If I give it a form, I can configure it once and then it knows what to do forever”

That means:

👉 You are building a **Form Logic Trainer**

NOT:

- a PDF viewer
 - a chatbot
 - a one-off tool
-

The correct product model

Mode 1: TRAIN

You define:

- Question: “History of hypertension?”
- Type: yes/no
- Required: yes
- If YES → ask:
 - medication?
 - controlled?
 - restrictions?

- If NO → skip

Save.

Done.

Mode 2: RUN

User sees:

Do you have a history of hypertension?

Clicks:

- Yes → follow-ups appear
- No → next question

System handles everything.

✗ Why I suggested split screen earlier

Because some tools try to visually match the form.

But you're right — that adds:

- complexity
- bugs
- wasted credits
- unnecessary UI work

You don't need it to prove value.

✓ The "do it right" version (your version)

You want:

- reusable templates ✓
- logic-driven questions ✓
- follow-up branching ✓
- adaptable to any form ✓

- no rework each time ✓

So here is the **correct MVP** definition:

YOUR FINAL MVP (clean + complete)

Features

1. Template Builder

- Create form template
 - Add sections
 - Add questions
 - Set:
 - answer type
 - required
 - follow-ups (rules)
 - Save template
-

2. Interview Runner

- One question at a time
 - Smart branching:
 - yes → follow-up
 - no → skip
 - Back / Next buttons
 - Progress indicator
 - Save answers
-

3. Review Page

- All answers grouped by section
 - Missing required fields
 - Flags
-

4. Reusability

- Use same template unlimited times
 - Duplicate templates
 - Edit templates
-

What you should NOT build (yet)

Do NOT include:

- PDF autofill
- drag-and-drop mapping
- AI extraction
- split screen
- file parsing
- fancy UI animations

All of that will kill your build on free tier.

What makes this "complete"

Not visuals.

Not AI.

 **Logic correctness + reusability**

If this works:

- You never manually think through forms again
- New hires don't need training
- No missed fields
- No inconsistent follow-ups

That's the real win.

The ONE thing you must get right

Your **question schema**

Every question must support:

- id
- text
- answer type
- required
- parent question
- trigger condition
- order

If that is solid → everything else works.



Your FINAL surgical Replit prompt

Paste this EXACTLY:

Build a web app called FormPath.



Purpose:

This app is a reusable form logic trainer and guided interview system. It allows

Core idea:

The app must NOT be hardcoded to any specific form. Instead, it must allow creating

Main features:

1. Template Builder

- Create a new form template
- Add sections to the form
- Add questions to each section
- Each question must support:
 - question text
 - answer type (yes/no, text, number, date, dropdown)
 - required (true/false)
 - display order
 - parent question (optional)
 - trigger value (e.g. show only if parent answer = yes)
- Allow nested follow-up questions
- Save template

2. Interview Runner

- Select a template and start a new session
- Show one question at a time

- If a question has follow-up questions triggered by the answer, show them
- If not triggered, skip them
- Allow back and next navigation
- Save answers in real time
- Show progress percentage

3. Review Page

- Show all answers grouped by section
- Highlight missing required fields
- Allow editing answers

4. Data storage

Use a simple database with:

- templates
- sections
- questions
- sessions
- answers

Important:

- The system must support unlimited templates
- Questions and logic must be reusable
- Do not include PDF handling or file parsing yet
- Focus on logic correctness and usability

UI:

- Clean professional interface
- Sidebar navigation
- Simple forms for building templates
- Minimal and fast

Tech:

- React frontend
- Node/Express backend
- SQLite or Replit DB

Also generate:

- basic database schema
- seed data with one example template showing branching logic

Final reality check

If you build THIS version successfully:

You already have:

- a scalable product
- something actually useful at Occu-Med
- something no one else in your workflow has

Everything else (PDF, AI, automation) becomes **add-ons later**.

If you want next, I can:

- design your **first actual template (DD2807-style logic)**
- or break this into **tiny credit-efficient prompts** so Replit doesn't mess it up

Just tell me 👍



I mean I obviously need it to save the form being completed or instantly emailing it back to me when they click complete so that it makes sense ...

It should also start out with a homepage that leads to two separate portals one for clinics and clinic personnel and the other for examinees.

Additionally, the UI should be MacOs Tahoe glass style and luminous with navy and blue colors.

It should also allow us to generate an invitation link to send it to applicants

Or create packets by selecting multiple forms and saving them as saved packets

You're thinking about this the right way now — this is no longer just a tool, this is a **real product**. The key is: we'll include your features, but keep the build **tight enough to**

actually work on free tier.

I'm going to give you a **clean, complete version** of what you want — not bloated, not half-baked.

What you're building (final form)

This is:

👉 **A form-template engine + guided intake system + packet generator**

with:

- reusable logic
- invitation links
- completion tracking
- email delivery
- role-based entry (clinic vs examinee)

That's legit.

FINAL PRODUCT STRUCTURE

1. Homepage (entry point)

Two portals:

Examinee Portal

- "Start your forms"
- Accepts invitation link
- Runs guided interview

Clinic Portal

- Internal use
- Create templates
- Build packets
- Send invites
- Review submissions

2. Template System (your “training” layer)

You create:

- forms
- sections
- questions
- logic

Saved forever.

3. Packet Builder (🔥 important)

This is a big upgrade — and VERY smart.

You can:

- combine multiple forms into one packet
- save it as:
 - “Deployment Packet”
 - “Dental Only”
 - “Labs Only”
- reuse forever

So instead of sending 5 forms → send 1 **guided experience**

4. Invitation System

You generate a link like:

/start/abc123



That link:

- is tied to a packet
 - optionally tied to a person
 - opens the guided interview
-

5. Interview Engine (core)

- one question at a time
 - branching logic
 - progress tracking
 - auto-save
 - clean UX
-

6. Completion Flow

When user clicks **Complete**:

- ✓ Save all answers
 - ✓ Mark session complete
 - ✓ Trigger email
 - ✓ Store submission for clinic
-

7. Email Output

MVP version (simple + reliable):

- send structured summary (NOT PDF yet)
- include:
 - name
 - packet name
 - all answers grouped by section

Later:

- attach PDF
 - attach formatted packet
-

UI STYLE (your requirement)

You said:

MacOS Tahoe glass, luminous, navy/blue

So define it clearly for Replit:

- dark navy base
- translucent panels (glass effect)
- soft blur backgrounds
- glowing accents (blue gradients)
- rounded cards
- minimal text clutter

This matters — you must explicitly tell it.

⚠️ IMPORTANT (free tier reality)

We need to:

- avoid overloading first build
- include features but keep them **simple implementations**

So:

Feature	MVP Version
Email	basic SMTP (no fancy templates)
Links	simple token system
Packets	just grouping templates
UI	styled but not over-engineered
Database	simple schema

🧠 DATA MODEL (expanded for your features)

You now need:

templates

- id
- name

sections

- id
- template_id

questions

- id
- section_id
- parent_question_id
- trigger_value

packets

- id
- name

packet_templates

- packet_id
- template_id

sessions

- id
- packet_id
- invite_token
- status (in_progress / complete)

answers

- session_id
- question_id
- value

THIS IS YOUR FINAL REPLIT PROMPT

This is the one you paste.

Build a web app called FormPath.



Purpose:

This is a reusable form logic system for Occu-Med style workflows. It allows bu

The system must support two main user flows: clinic users and examinees.

HOME PAGE

Create a clean landing page with two portals:

1. Examinee Portal
 - Button: "Start Your Forms"
 - Allows access via invitation link
2. Clinic Portal
 - Button: "Clinic Login"
 - Leads to dashboard

CLINIC PORTAL FEATURES

Dashboard:

- View templates
- View packets
- View submitted sessions

Template Builder:

- Create form templates
- Add sections
- Add questions

Each question must support:

- question text
- answer type (yes/no, text, number, date, dropdown)
- required (true/false)
- display order
- parent question (optional)
- trigger value (show only if parent answer matches)

Support nested follow-up questions.

PACKET BUILDER

- Create a packet

- Select multiple templates
 - Save packet for reuse
-

INVITATION SYSTEM

- Generate unique invitation link for a packet
 - Each link creates a session
 - Link format example: /start/{token}
 - Store token in database
-

EXAMINEE FLOW

- User opens invitation link
 - Starts guided interview
 - Questions shown one at a time
 - If answer triggers follow-ups, show them
 - Otherwise skip them
 - Save answers in real time
 - Show progress bar
 - Allow back and next navigation
-

COMPLETION

When user clicks "Complete":

- Save session as complete
 - Store all answers
 - Send email notification with submission summary
-

EMAIL SYSTEM

- Use simple email sending (SMTP or nodemailer)
 - Send structured summary:
 - packet name
 - sections
 - questions and answers
-

REVIEW SYSTEM (Clinic Portal)

- View completed sessions
 - Expand to see answers grouped by section
 - Highlight missing required fields
-

DATA MODEL

Use database tables for:

- templates
 - sections
 - questions
 - packets
 - packet_templates
 - sessions
 - answers
-

UI DESIGN

Style must match:

- MacOS Tahoe glass effect
 - Dark navy background
 - Translucent panels with blur
 - Blue glowing accents
 - Rounded cards
 - Clean, modern internal tool look
-

TECH

- React frontend
 - Node/Express backend
 - SQLite or Replit DB
-

IMPORTANT

- Do NOT hardcode any specific form
- Must support unlimited templates and packets
- Focus on logic, usability, and reliability
- Keep implementation simple but complete

🌟 What you'll have after this works

If this builds correctly, you'll have:

- a working internal system
- reusable form logic
- invite-based intake
- standardized data collection
- zero "what do I ask next?" confusion

That's already **insanely valuable** for your workflow.

🚀 If you want next step

After this builds, I can help you add:

- PDF auto-fill (properly, not hacky)
 - smart contradiction detection (VERY useful for RDQA)
 - clinic-facing simplified packets
 - "missing component" auto flags (this is huge for you)
-

Just say:

👉 "optimize next step"

and I'll walk you through upgrading this without breaking it.

📷 📄 ⬆️ ↺ ...

No the entry point clinic section is not correct.. don't forget that we (occu-med) is the TPA .. the clinic section would be to send the clinic the forms to complete

Yes — that's an important correction.

Occu-Med should be the **admin/control layer**, not one of the public portals.

So the entry structure should really be:

Correct entry structure

1. Occu-Med Admin Portal

This is for **your team only**.

This is where Occu-Med can:

- create forms
- build packets
- create invitation links
- choose whether the packet is for an **examinee** or a **clinic**
- review submitted responses
- trigger email delivery
- save reusable templates and saved packets

2. Examinee Portal

This is for the **applicant / employee / examinee** to fill out their part.

3. Clinic Portal

This is for the **clinic or clinic personnel** to complete the forms Occu-Med sends them.

That makes much more sense because Occu-Med is acting as the coordinator/TPA in the middle.

The actual workflow should be

Occu-Med admin logs in and does one of these:

- sends an **examinee packet**
- sends a **clinic packet**
- sends both separately if needed

Example:

- Examinee gets: medical history questionnaire, demographics, signatures
- Clinic gets: exam forms, provider-completed sections, testing forms, attachments checklist

So the packet system should support assigning a packet to a **recipient type**:

- **examinee**
- **clinic**

That way the app knows who the form experience is for.

Better homepage structure

The landing page should have **three paths**, not two:

A. Occu-Med Admin

Internal dashboard for your team

B. Examinee Access

Access via secure invitation link

C. Clinic Access

Access via secure invitation link

Or even better:

- Public homepage is simple
- Occu-Med admin signs in separately
- Examinee and clinic usually enter through their invitation links

That is probably the cleanest.

Better product model

The system should now have **3 roles**:

Occu-Med Admin

Can:

- create/edit templates
- create/edit packets
- generate secure links
- send forms
- monitor completion
- review answers
- resend reminders

- receive completed submissions

Examinee

Can:

- open their link
- complete assigned packet
- save progress
- submit

Clinic User

Can:

- open their link
- complete assigned provider/clinic packet
- save progress
- submit back to Occu-Med

What needs to change in the packet builder

Each packet should include:

- packet name
- recipient type
- included forms
- email destination
- optional instructions

So for example:

Packet Name: Fort Bliss Examinee Intake

Recipient Type: Examinee

Packet Name: Fort Bliss Clinic Completion Packet

Recipient Type: Clinic

That distinction is huge.

What the clinic portal should really do

The clinic portal is **not** for managing templates or internal dashboards.

It is just for:

- opening assigned packet
- completing requested forms
- uploading any required results or attachments later if you add that
- submitting back to Occu-Med

So it should feel more like a secure completion portal, not an admin system.

Revised core structure

Here is the correct structure:

Occu-Med Admin Portal

- Manage templates
- Manage packets
- Create saved packets
- Generate invitation links
- Send email invitations
- Track packet status
- Review submissions

Examinee Portal

- Complete assigned packet
- Save progress
- Submit

Clinic Portal

- Complete assigned packet
- Save progress
- Submit

Revised logic for invitation links

When Occu-Med creates a link, the system should store:

- packet id
- recipient type (clinic or examinee)
- token
- recipient email
- status
- created date

- completed date

Then when the link opens, the app automatically knows which experience to show.

Your corrected master prompt

Use this instead:

Build a web app called FormPath. 

Purpose:

This app is a reusable form logic and packet delivery system for Occu-Med. Occu

Core structure:

The app must support 3 user roles:

1. Occu-Med Admin
2. Examinee
3. Clinic User

```
=====
OCCU-MED ADMIN PORTAL
=====
```

This is the internal dashboard for Occu-Med staff.

Features:

- Create and manage form templates
- Add sections and questions
- Configure question logic
- Create and save reusable packets
- Select multiple forms and save them as one packet
- Assign a packet recipient type:
 - examinee
 - clinic
- Generate secure invitation links
- View all sessions and statuses
- Review completed submissions
- Send email notifications on completion

```
=====
FORM TEMPLATE BUILDER
=====
```

Allow Occu-Med admin to create reusable form templates.

Each template must support:

- form name
- sections
- questions
- answer type (yes/no, text, number, date, dropdown)
- required true/false
- display order
- parent question (optional)
- trigger value (optional)

Support nested follow-up questions.

=====

PACKET BUILDER

=====

Allow Occu-Med admin to:

- create packet
- name packet
- select multiple form templates
- save packet for reuse
- choose recipient type:
 - examinee
 - clinic

=====

INVITATION SYSTEM

=====

Allow Occu-Med admin to generate secure invitation links tied to:

- packet
- recipient type
- recipient email
- session record

Link format example:

/start/{token}

=====

EXAMINEE PORTAL

=====

Examinees access their assigned packet through a secure invitation link.

Features:

- guided interview flow
- one question at a time
- branching follow-up logic

- save progress
- back and next navigation
- progress bar
- submit packet

CLINIC PORTAL

Clinics access their assigned packet through a secure invitation link.

Features:

- guided form completion flow
- one question at a time
- branching follow-up logic if configured
- save progress
- back and next navigation
- progress bar
- submit packet back to Occu-Med

COMPLETION BEHAVIOR

When a clinic or examinee clicks Complete:

- save the session as complete
- store all answers
- notify Occu-Med by email
- include a structured summary of answers

REVIEW SYSTEM

Occu-Med admin can:

- view all sessions
- filter by clinic or examinee
- see status:
 - not started
 - in progress
 - completed
- open completed submissions
- view answers grouped by section
- highlight missing required fields

DATA MODEL

=====

Use tables for:

- users
- templates
- sections
- questions
- packets
- packet_templates
- invitations
- sessions
- answers

=====

UI DESIGN

=====

Use a MacOS Tahoe-inspired glass style:

- dark navy and deep blue background
- translucent glass panels
- blur effects
- blue luminous accents
- rounded cards
- modern polished interface

=====

TECH

=====

- React frontend
- Node/Express backend
- SQLite or Replit DB
- simple email sending with nodemailer

=====

IMPORTANT

=====

- Occu-Med is the internal admin and coordinator
- clinic users are external recipients, not administrators
- examinees are external recipients, not administrators
- the system must support unlimited reusable templates and packets
- focus on reliability and complete workflow

One more improvement you should add

Tell it to support **saved packet types by audience**, like:

- "Examinee Packet"
- "Clinic Packet"
- "Combined Workflow Set"

That will make reuse much easier later.

The simplest way to think about it now

This is no longer:

"a clinic portal and an examinee portal."

It is:

"an Occu-Med admin system that dispatches guided packets to two different external audiences."

That framing is the correct one.

If you want, I can now rewrite this into an even more credit-efficient version that is tighter and less likely to confuse Replit.



Yes

Absolutely — here's the **tighter, cleaner, more credit-efficient Replit prompt**.

This version is written to reduce confusion and keep Replit focused on the real workflow.

Build a web app called FormPath.



PURPOSE

FormPath is an Occu-Med admin system for creating reusable form templates, com

Occu-Med is the administrator and coordinator.

There are 3 roles in the system:

1. Occu-Med Admin
2. Examinee
3. Clinic User

The app must support reusable templates and reusable packets.
It must not be hardcoded to one form.

=====

CORE WORKFLOW

=====

1. Occu-Med Admin creates form templates
2. Occu-Med Admin combines templates into packets
3. Occu-Med Admin chooses recipient type:
 - examinee
 - clinic
4. Occu-Med Admin generates a secure invitation link
5. Recipient opens the link and completes the packet
6. On completion, the system saves answers and emails Occu-Med a summary

=====

ROLE 1: OCCU-MED ADMIN PORTAL

=====

Create a secure internal admin portal for Occu-Med staff.

Admin portal must include:

A. Dashboard

- View form templates
- View saved packets
- View invitation sessions
- View submission status

B. Template Builder

- Create a form template
- Add sections
- Add questions to sections

Each question must support:

- question text
- answer type:
 - yes/no
 - short text
 - long text
 - number
 - date
 - dropdown
- required true/false
- display order
- parent question (optional)

- trigger value (optional)

Rules:

- If a question has a parent question and a trigger value, only show it when t
- Must support nested follow-up questions

C. Packet Builder

- Create packet
- Name packet
- Select multiple form templates
- Save packet for reuse
- Set recipient type:
 - examinee
 - clinic

D. Invitation Manager

- Generate secure invitation link for a packet
- Associate link with:
 - packet
 - recipient type
 - recipient email
 - status
- Invitation link format:
/start/{token}

E. Review Submissions

- View all sessions
- Filter by:
 - recipient type
 - status
- Status values:
 - not started
 - in progress
 - completed
- Open a completed session
- View answers grouped by section
- Highlight missing required answers

ROLE 2: EXAMINEE PORTAL

Examinees do not have admin access.

They only access their assigned packet through a secure invitation link.

Examinee flow:

- Open invitation link

- Start packet
- Complete questions one at a time
- Save answers automatically
- See progress bar
- Use Back and Next buttons
- Submit when finished

Branching behavior:

- If answer triggers follow-up questions, show them
- If not triggered, skip them

=====

ROLE 3: CLINIC PORTAL

=====

Clinic users do not have admin access.

They only access their assigned packet through a secure invitation link.

Clinic flow:

- Open invitation link
- Start packet
- Complete questions one at a time
- Save answers automatically
- See progress bar
- Use Back and Next buttons
- Submit when finished

Branching behavior:

- If answer triggers follow-up questions, show them
- If not triggered, skip them

=====

COMPLETION LOGIC

=====

When an examinee or clinic user clicks Complete:

- Save the session as completed
- Save all answers in the database
- Send an email notification to Occu-Med
- Email must contain:
 - recipient type
 - packet name
 - submission status
 - answers grouped by section

=====

DATA MODEL

=====

Use database tables for:

users

- id
- email
- password_hash
- role

templates

- id
- name
- description
- created_at

sections

- id
- template_id
- title
- display_order

questions

- id
- section_id
- question_text
- answer_type
- required
- display_order
- parent_question_id
- trigger_value

packets

- id
- name
- recipient_type
- created_at

packet_templates

- id
- packet_id
- template_id

invitations

- id
- packet_id
- recipient_type

- recipient_email
- token
- status
- created_at

sessions

- id
- invitation_id
- packet_id
- recipient_type
- status
- started_at
- completed_at

answers

- id
- session_id
- question_id
- answer_value
- saved_at

UI / DESIGN

Style the app in a MacOS Tahoe-inspired glass look.

Requirements:

- dark navy and blue color palette
- translucent glass panels
- soft blur effects
- luminous blue accents
- rounded cards
- polished modern interface
- desktop-first layout
- clean and uncluttered

TECH

Use:

- React frontend
- Node/Express backend
- SQLite or Replit database
- Nodemailer for email sending

```
=====
MVP LIMITS
=====
```

For the first build:

- Do not implement PDF autofill yet
- Do not implement AI extraction
- Do not implement drag-and-drop field mapping
- Focus on:
 - reusable templates
 - reusable packets
 - invitation links
 - guided completion
 - answer saving
 - completion email

```
=====
SEED DATA
=====
```

Include seed data with:

- 1 example template
- 2 sections
- a few questions
- at least 1 yes/no follow-up branching example
- 1 example packet using that template

```
=====
IMPORTANT
=====
```

Occu-Med is the internal admin and coordinator.
 Clinic users and examinees are external recipients only.
 The app must support unlimited reusable templates and packets.
 The app must be reliable and logically correct.

Use it in this order so you do not waste credits:

First paste only that prompt.

If Replit gives you a rough first version, your next follow-up should be:

Improve the packet invitation flow.



Requirements:

- From the admin dashboard, allow me to create an invitation by selecting a pac

- Store invitation status as not started, in progress, or completed.
- Make the invitations list easy to scan from the dashboard.

Then after that:

Improve the guided interview runner.



Requirements:

- Show one question at a time
- Support nested follow-up questions reliably
- Autosave every answer
- Show progress bar
- Prevent completion if required visible questions are unanswered

Then after that:

Improve the UI styling.



Requirements:

- Stronger MacOS Tahoe glass aesthetic
- Dark navy background
- More luminous blue glow
- Better spacing
- More polished translucent cards
- Cleaner typography and buttons

Then after that:

Improve the completion flow.



Requirements:

- When the user clicks Complete, save the session as completed
- Send an email summary to Occu-Med
- The summary should group answers by section
- Show a clean confirmation screen after successful submission

The reason this order matters is that it forces Replit to build the **logic first**, then the polish.

If you want, I can do one more thing next: write the **actual first sample template structure** for an Occu-Med packet so you can seed it with something real instead of generic demo questions.



Why?

For the first build:

- Do not implement PDF autofill yet
- Do not implement AI extraction
- Do not implement drag-and-drop field mapping
- Focus on:
 - reusable templates
 - reusable packets
 - invitation links
 - guided completion
 - answer saving
 - completion email

If it can do these this is what I want it make only give me one shot to do it due to the nature of the request.

Also the completed form must be emailed back to occu-med not a summary.

Third lets tell it to make it as hipaa compliant as possible.

Fourth, lets make it so that they can digitally sign the forms as well and they need to click a validation check to make sure no answers or sections were missed.

Because those features make the build much heavier and more failure-prone on a one-shot prompt.

But since you're saying this is your **one chance**, then the better move is **not** to hold them back. The better move is to ask for the full product in one pass, while being very explicit about priorities, constraints, and what "done" means.

The one thing I would not let Replit overpromise on is **HIPAA compliance**. It can build toward **HIPAA-conscious / best-practice protections**, but no prompt can magically make an app legally HIPAA compliant by itself. That depends on hosting, access controls, audit logs, encryption, BAAs, email handling, storage practices, and your actual operational setup. So the prompt should say:

- build it to be **as HIPAA-conscious as possible**
- use strong privacy/security safeguards
- minimize PHI exposure
- do not claim certified compliance
- structure it so it can be hardened for compliance

Also, if you want the **completed forms emailed back**, say that clearly:

- email the **completed packet / completed form output**
- not just a summary

And yes, add:

- **digital signatures**
- **final validation checklist**
- user must confirm nothing was skipped before submission

Here is the revised **single-shot master prompt**:

Build a web app called FormPath.



PURPOSE

FormPath is an Occu-Med admin system for creating reusable form templates, com

Occu-Med is the administrator and third-party coordinator (TPA).

There are 3 roles in the system:

1. Occu-Med Admin
2. Examinee
3. Clinic User

The app must support unlimited reusable templates and packets.

It must not be hardcoded to one specific form.

It must be designed so Occu-Med can train new forms and reuse them repeatedly.

=====

CORE WORKFLOW

=====

1. Occu-Med Admin creates form templates

2. Occu-Med Admin combines templates into packets
3. Occu-Med Admin chooses recipient type:
 - examinee
 - clinic
4. Occu-Med Admin generates a secure invitation link
5. Recipient opens the secure link and completes the assigned packet
6. Recipient digitally signs where required
7. Recipient completes a final validation confirmation before submission
8. On submission, the system:
 - saves all answers
 - marks the session complete
 - generates a completed packet / completed form output
 - emails the completed packet back to Occu-Med

=====

ROLE 1: OCCU-MED ADMIN PORTAL

=====

Create a secure internal admin portal for Occu-Med staff.

Admin portal must include:

A. Dashboard

- View form templates
- View saved packets
- View invitation sessions
- View submission status
- View completed submissions
- Search and filter invitations and submissions

B. Template Builder

- Create a form template
- Upload and attach a source form file (PDF) for reference and later output ger
- Add sections
- Add questions to sections

Each question must support:

- question text
- answer type:
 - yes/no
 - short text
 - long text
 - number
 - date
 - dropdown
 - checkbox
 - signature

- required true/false
- display order
- parent question (optional)
- trigger value (optional)
- helper text/instructions
- whether it belongs to examinee flow or clinic flow if needed
- whether it requires signature or attestation

Rules:

- If a question has a parent question and trigger value, only show it when the
- Must support nested follow-up questions
- Must support conditional hiding/skipping of irrelevant questions
- Must support required field validation

C. Packet Builder

- Create packet
- Name packet
- Select multiple form templates
- Save packet for reuse
- Set recipient type:
 - examinee
 - clinic
- Allow saved packet presets such as:
 - examinee packet
 - clinic packet
 - deployment packet
 - dental packet
 - labs packet

D. Invitation Manager

- Generate secure invitation link for a packet
- Associate link with:
 - packet
 - recipient type
 - recipient email
 - status
- Invitation link format:
/start/{token}
- Allow copying the invitation link
- Allow emailing the invitation link from the app

E. Review Submissions

- View all sessions
- Filter by:
 - recipient type
 - status
 - packet

- recipient email
- Status values:
 - not started
 - in progress
 - completed
- Open a completed session
- View answers grouped by section
- View attached completed packet output
- View signature confirmations
- Highlight missing required answers if any
- Allow internal review notes

=====

ROLE 2: EXAMINEE PORTAL

=====

Examinees do not have admin access.

They only access their assigned packet through a secure invitation link.

Examinee flow:

- Open invitation link
- Start packet
- Complete questions one at a time
- Save answers automatically
- See progress bar
- Use Back and Next buttons
- See clear required-field indicators
- Sign digitally where required
- Complete final validation step before submission
- Submit when finished

Branching behavior:

- If answer triggers follow-up questions, show them
- If not triggered, skip them

=====

ROLE 3: CLINIC PORTAL

=====

Clinic users do not have admin access.

They only access their assigned packet through a secure invitation link.

Clinic flow:

- Open invitation link
- Start packet
- Complete questions one at a time
- Save answers automatically

- See progress bar
- Use Back and Next buttons
- See clear required-field indicators
- Sign digitally where required
- Complete final validation step before submission
- Submit when finished

Branching behavior:

- If answer triggers follow-up questions, show them
- If not triggered, skip them

=====

DIGITAL SIGNATURES

=====

Implement digital signature support.

Requirements:

- Signature field type must be supported
- Allow drawing signature with mouse or touch
- Store signature image or signature data securely
- Associate signature with:
 - session
 - signer role
 - timestamp
- Show signed status in the admin portal
- Allow signature fields to be marked required before submission

=====

FINAL VALIDATION STEP

=====

Before the user can submit, require a final validation screen.

Requirements:

- Show completion checklist
- Confirm all required visible questions are answered
- Confirm all required signature fields are completed
- Require user to check a confirmation box such as:
 - "I confirm that I reviewed this packet and completed all required answers and
- Prevent submission until this confirmation is checked
- If anything required is missing, show clear blocking errors

=====

COMPLETION LOGIC

=====

When an examinee or clinic user clicks Complete:

- Save the session as completed
- Save all answers in the database
- Save all signature data
- Generate a completed packet / completed form output
- Email the completed packet back to Occu-Med
- Show a clean confirmation screen after successful submission

IMPORTANT:

Do not send only a summary email.

The system must email the completed packet or completed form output back to Occu-Med

=====

COMPLETED PACKET OUTPUT

=====

Create a completed output when a packet is submitted.

Requirements:

- Generate a completed version of the packet using the user's answers
- Include all answered fields in a clean readable format
- Include signature fields where captured
- Prefer generating a PDF output if feasible
- If direct PDF filling is difficult, generate a polished completed packet document
- Attach the completed output to the email sent to Occu-Med
- Also save the completed output in the admin portal for later review/download

=====

EMAIL SYSTEM

=====

Use email sending functionality.

Requirements:

- Occu-Med Admin can send invitation links by email
- On completion, email the completed packet output back to Occu-Med
- Include session metadata in the email:
 - recipient type
 - packet name
 - recipient email
 - completion timestamp
- Attach the completed packet file
- Keep the email professional and minimal
- Do not include unnecessary PHI in the subject line

=====

HIPAA-CONSCIOUS / SECURITY-FOCUSED REQUIREMENTS

=====

Build the app to be as privacy-conscious and HIPAA-conscious as possible.

IMPORTANT:

Do not claim the app is legally HIPAA compliant.

Instead, implement strong best-practice safeguards and structure the system for

Requirements:

- Role-based access control
- Secure authentication for Occu-Med admin users
- Token-based secure invitation links
- Minimize exposure of PHI in URLs, logs, and email subject lines
- Encrypt sensitive data at rest where feasible
- Use HTTPS-ready architecture
- Store passwords securely with hashing
- Add audit-style tracking for:
 - invitation creation
 - session start
 - session completion
 - signature completion
- Prevent public access to admin functions
- Session timeout / secure session handling where feasible
- Avoid exposing unnecessary PHI in frontend lists
- Structure file storage so completed packets can be access-controlled
- Add clear placeholders/config areas for secure environment variables:
 - email credentials
 - app secret
 - database connection secrets
- Write code and comments in a way that makes later hardening easier

=====

DATA MODEL

=====

Use database tables for:

users

- id
- email
- password_hash
- role
- created_at

templates

- id
- name

- description
- source_form_file_path
- created_at

sections

- id
- template_id
- title
- display_order

questions

- id
- section_id
- question_text
- answer_type
- required
- display_order
- parent_question_id
- trigger_value
- helper_text
- requires_signature

packets

- id
- name
- recipient_type
- created_at

packet_templates

- id
- packet_id
- template_id

invitations

- id
- packet_id
- recipient_type
- recipient_email
- token
- status
- created_at
- started_at
- completed_at

sessions

- id
- invitation_id

- packet_id
- recipient_type
- status
- started_at
- completed_at
- final_validation_confirmed

answers

- id
- session_id
- question_id
- answer_value
- saved_at

signatures

- id
- session_id
- question_id
- signer_role
- signature_data
- signed_at

completed_outputs

- id
- session_id
- file_path
- file_type
- created_at

audit_logs

- id
- user_or_session_ref
- event_type
- event_details
- created_at

UI / DESIGN

Style the app in a MacOS Tahoe-inspired glass look.

Requirements:

- dark navy and blue color palette
- translucent glass panels
- soft blur effects
- luminous blue accents

- rounded cards
- polished modern interface
- desktop-first layout
- clean and uncluttered
- premium, sleek, medical-admin feel

=====

TECH

=====

Use:

- React frontend
- Node/Express backend
- SQLite or Replit database
- Secure environment variable usage
- Nodemailer or equivalent email handling
- PDF/document generation support for completed packet output
- Signature capture component

=====

BUILD PRIORITIES

=====

Prioritize these features as essential:

1. reusable templates
2. reusable packets
3. secure invitation links
4. guided completion flow
5. autosaving answers
6. digital signatures
7. final validation confirmation
8. completed packet output generation
9. emailing the completed packet back to Occu-Med
10. privacy-conscious and HIPAA-conscious architecture

=====

SEED DATA

=====

Include seed data with:

- 1 example template
- 2 sections
- several sample questions
- at least 1 yes/no branching example
- at least 1 signature field example
- 1 example packet using that template

=====

IMPORTANT

=====

Occu-Med is the internal admin and coordinator.
Clinic users and examinees are external recipients only.
The app must support unlimited reusable templates and packets.
The app must be logically correct, usable, and as complete as possible in one k
If any feature must be approximated in MVP form, preserve the architecture so d

A few things to keep in mind before you use it:

- **Emailing actual completed forms** is reasonable, but if the original PDFs are not field-fillable, Replit may generate a clean completed PDF packet rather than perfectly writing back into the exact original form layout.
- **HIPAA-conscious** is a good requirement. **"HIPAA compliant"** is not something I'd tell it to claim.
- **Digital signatures** are realistic for capture and storage. Whether they satisfy every legal/evidentiary use case is separate, but for workflow signatures this is the right ask.

The only extra line I'd add after it builds, if needed, is:

Make sure submission is blocked unless all required visible questions, required 

If you want, I can also make this into an even more aggressive **"CEO-grade one-shot prompt"** version with less explanatory text and more direct build commands.

    